

Predavanje 1 — Uvod u paralelno programiranje

Paralelno programiranje

Damir Krstinić Antonia Ivanda

FESB — Sveučilište u Splitu

1. **O kolegiju**

Općenito o kolegiju

- ▶ **Predavanja:** prof. dr. sc. Damir Krstinić; doc. dr. sc. Antonia Ivanda
- ▶ **Laboratorijske vježbe:** univ. mag. ing. comp. Jakov Bejo
- ▶ Način ocjenjivanja definiran je u sustavu *FESB Nastava*

Sadržaj kolegija

- ▶ **Dio I** — uvod i teorijska razmatranja
- ▶ **Dio II** — arhitekture s dijeljenom memorijom (višenitno programiranje, OpenMP)
- ▶ **Dio III** — arhitekture s distribuiranom memorijom (MPI)
- ▶ **Dio IV** — GPU arhitekture (CUDA)

2.

Zašto paralelno računanje?

- ▶ Cilj: riješiti problem **brže** nego serijskim (slijednim) računanjem
- ▶ Može li rast računalne snage pratiti sve zahtjevnije probleme?
- ▶ **FLOPS** — *Floating Point Operations per Second*:
 - ▶ 1950.: oko 10^2 FLOPS
 - ▶ danas: preko 10^{18} FLOPS (eksaskala)

Mooreov zakon: broj tranzistora u integriranom krugu udvostručuje se otprilike svake dvije godine.

Gordon Moore — izvorni zakon i revizija

- ▶ **1965.** — Moore predviđa udvostručenje broja komponenti po integriranom krugu **svake godine**
 - ▶ trend počinje izumom integriranog kruga 1958.
- ▶ **1975.** — revidira prognozu: udvostručenje **svake dvije godine**

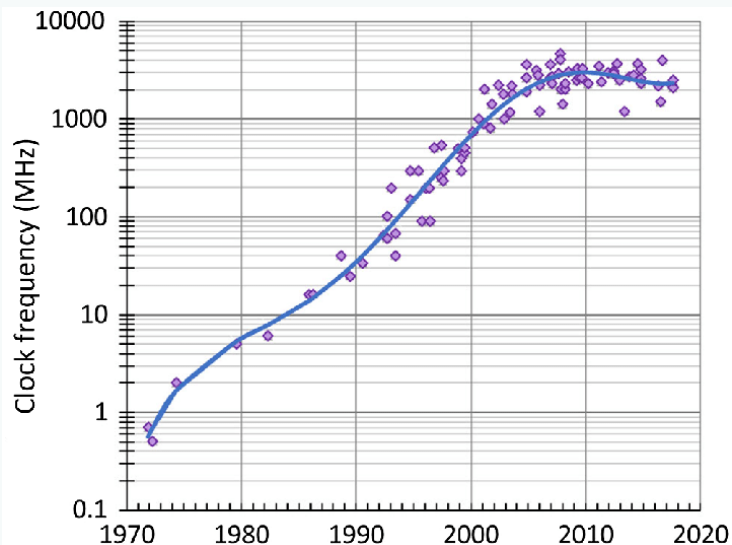
Our World
in DataOur World
in Data

Licensed under [CC-BY-SA](#) by the author Max Roser.

Fizikalni limiti brzine

- ▶ Brzina računala određena je trajanjem jednog **ciklusa**
- ▶ Trajanje ciklusa ograničeno je fizikom:
 - ▶ brzina svjetlosti: $3 \cdot 10^8$ m/s (≈ 30 cm/ns)
 - ▶ brzina signala u bakrenom vodiču: ≈ 9 cm/ns
- ▶ Frekvencija procesora prestala je rasti — dolazimo do **zida** frekvencije i snage

Frekvencija procesora kroz vrijeme



Granice minijaturizacije

- ▶ Osim brzine, ograničen je i **broj tranzistora** na pločici
- ▶ Tranzistori se smanjuju do razmjera **nanometara**:
 - ▶ današnji proizvodni procesi reda su nekoliko nm
 - ▶ vrata tranzistora široka su tek desetak atoma (atom silicija $\approx 0,2$ nm)
- ▶ Na tim razmjerima javljaju se **kvantni efekti**:
 - ▶ tuneliranje elektrona $\rightarrow$ struja curenja
 - ▶ porast gustoće topline
- ▶ Fizička veličina atoma i vodiča postavlja **donju granicu** — Mooreov zakon usporava
- ▶ Dobitak se zato sve više traži u **paralelizmu**, a ne u sve manjim tranzistorima

Odgovor: korištenje višestrukih resursa

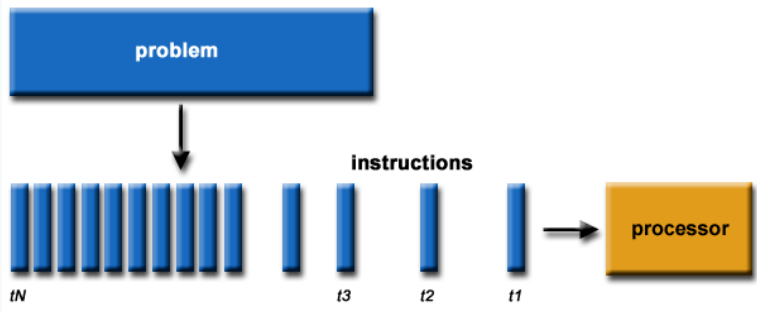
- ▶ Paralelizacija na razini **sklopovlja**:
 - ▶ višejezgreni procesori (dual / quad / octa / ...)
 - ▶ pipelining, hyperthreading, FPGA, GPU
- ▶ Korištenje **višestrukog** sklopovlja:
 - ▶ *cluster* — homogeno sklopovlje
 - ▶ *grid* — heterogeno, prostorno distribuirano
 - ▶ *cloud* — klijent/poslužitelj, visoka dostupnost usluga

3.

Serijsko i paralelno računanje

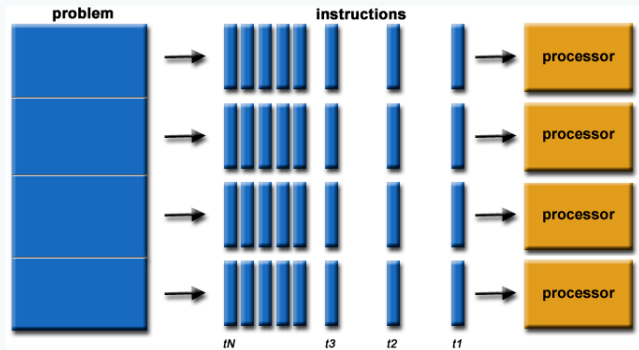
Serijsko (slijedno) računanje

- ▶ Problem se rastavlja na male diskretne korake (instrukcije)
- ▶ Instrukcije se izvršavaju **jedna za drugom**, redom kako su navedene
- ▶ Izvršavanje na **jednom** procesoru — u svakom trenutku jedna instrukcija
- ▶ Neovisno o programskoj paradigmi (proceduralno, objektno ...)



Paralelno računanje

- ▶ Istovremeno korištenje **višestrukih resursa** za brže rješavanje složenih problema
- ▶ Problem se rastavlja na dijelove koji se izvršavaju **istovremeno**
- ▶ Svaki dio dalje se dijeli na slijedne korake



Komunikacija i kombiniranje rezultata

- ▶ Dijelovi koji se izvršavaju istovremeno često dijele isti ili sličan kod
- ▶ Međusobno **komuniciraju**:
 - ▶ putem **zajedničke (dijeljene) memorije**
 - ▶ **razmjenom poruka**
- ▶ Po završetku se rezultati **kombiniraju** u konačni rezultat

4.

Znanje, algoritmi i složenost

Deklarativno i imperativno znanje

- ▶ **Deklarativno znanje** — činjenice, što je istina:
 - ▶ „jabuka je voće”
 - ▶ „y je korijen iz x samo ako je $y \cdot y = x$ ”
- ▶ **Imperativno (proceduralno) znanje** — kako se zadatak rješava, *recept*:
 - ▶ često ga je teško opisati; velik dio usvajamo i nesvjesno

Algoritam

- ▶ Konačna, jednoznačna kombinacija primitivnih operacija koje alat podržava, izvršenih određenim redoslijedom — od ulaznih do izlaznih uvjeta
- ▶ Svojstva:
 - ▶ **konačnost** — završava u konačno koraka
 - ▶ **definitivnost** — određen skup ulaza i izlaza
 - ▶ **nedvosmislenost** — precizan skup i redoslijed operacija
 - ▶ **kompleksnost** — mjera složenosti izvršavanja

Složenost algoritma

- ▶ Analiza složenosti = određivanje količine potrebnih **resursa**
- ▶ **Vremenska** složenost: vrijeme kao funkcija veličine ulaznog skupa
- ▶ **Memorijska** složenost: prostor kao funkcija veličine ulaznog skupa
- ▶ Mjereno vrijeme ovisi o sklopovlju i ulazu → tražimo mjeru **neovisnu o stroju**

Primjer: bubble sort

```
n = length(A)           % n operacija
for i = n:-1:1          % vanjska petlja
    for j = 1:i-1        % (n-1)+(n-2)+... = n(n-1)/2
        if A(j) > A(j+1) % broj zamjena ovisi o
            zamijeni A(j), A(j+1) % redoslijedu brojeva
```

- Ukupan broj operacija proporcionalan je s n^2

Rast vremena izvršavanja

- ▶ Ako sortiranje 100 brojeva traje 5 s:
 - ▶ 200 brojeva → **20 s** (4× duže)
 - ▶ 300 brojeva → **45 s** (9× duže)
- ▶ Vrijeme raste s kvadratom veličine ulaza — $O(n^2)$

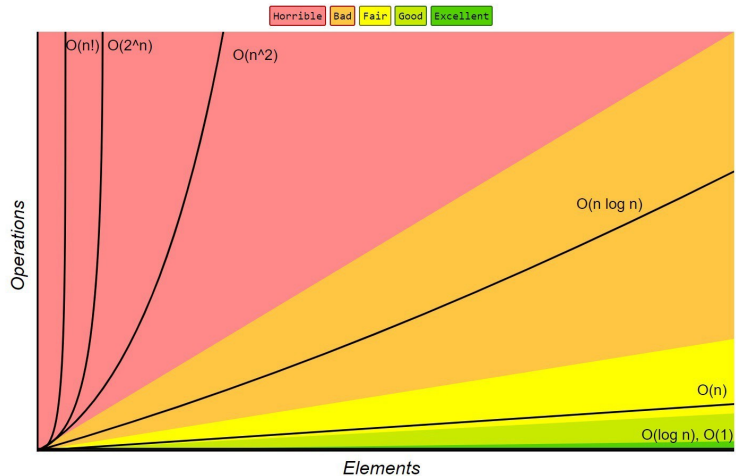
Big-O notacija

- ▶ Procjena složenosti u odnosu na veličinu ulaza N , **neovisno o sklopovlju**
- ▶ Pravila: zanemaruju se konstante i članovi nižeg reda
 - ▶ $O(9N) \rightarrow O(N)$
- ▶ Poredak klasa:

$$O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(2^N) < O(N!)$$

Klase složenosti

Big-O Complexity Chart



5. **Sažetak**

Ključne poruke

- ▶ Serijska brzina je fizikalno ograničena → budućnost je **paralelizam**
- ▶ Paralelno = istovremeno korištenje više resursa + komunikacija + kombiniranje
- ▶ Složenost algoritma (Big-O) određuje isplati li se i kako paralelizirati

Što slijedi i zadaci

- ▶ Sljedeće predavanje: **paralelna računala i arhitekture** (Flynnova taksonomija, memorijski modeli)
- ▶ Zadatak za samostalni rad — odredi klasu složenosti za:
 - ▶ izračun sume niza
 - ▶ izračun sume prvog i posljednjeg elementa niza